

UNITREE L1 ROS 2 PROJECT

File Types and Their Roles

A practical guide to the source code, configuration, build, launch, documentation and generated data used by the project

Project	Unitree L1 4D LiDAR integration
Runtime	Ubuntu 22.04 / ROS 2 Humble inside Docker
Main tools	colcon, RViz2, TF2, rosbag2 and OctoMap
Purpose	Understand which file controls each layer of the system
Scope	Project-specific examples plus general ROS 2 conventions

Core idea

Code implements what is possible.
Build files turn code into ROS packages.
Launch files decide what runs.
YAML files decide with which values it runs.
RViz files decide how the data is displayed.
Bags and maps contain the resulting data.
Documentation explains and proves the process.

Prepared from the Unitree L1 engineering workflow used in this project.

1. How the project is organised

The project is not one single program. It is a stack of files, each interpreted by a different tool.

```

Documentation
  ↓
Docker environment
  ↓
ROS package and build definitions
  ↓
Launch orchestration
  ↓
Node parameters
  ↓
C++ and Python programs
  ↓
LiDAR topics, bags, maps and logs
  
```

The extension is not enough

Two files can both use YAML syntax and still have completely different meanings. A ROS node parameter file, a Docker Compose file and colcon_defaults.yaml are all YAML, but each is read by a different program.

Five questions to ask when you find a file

1	Who reads it?	Docker, colcon, ROS launch, a ROS node, RViz, vcs, Python, CMake or a human.
2	Is it source or generated?	Edit src/ and project files; avoid editing build/, install/, caches, bags or maps by hand.
3	Does it control behavior or only appearance?	A node YAML changes runtime values; an .rviz file changes visualisation.
4	Does it require rebuilding?	C++ and package metadata usually do; many existing linked Python/config files only need a restart.
5	How will I verify it?	Query the effective parameter, inspect the graph, run tests, check topic rates or reopen the saved output.

2. Quick-reference table

Type	Examples	Read by / used by	Main purpose
<code>.md</code>	README.md, docs/*.md	Humans, Git platforms	Instructions, decisions, evidence and safety
<code>.pdf</code>	Engineering manual, report	Humans, PDF viewers	Stable printable documentation
<code>Dockerfile</code>	docker/Dockerfile	Docker build	Build the Ubuntu/ROS image
<code>Compose .yaml</code>	compose.yaml, compose.gui.yaml	Docker Compose	Create the runtime and expose GUI/hardware
<code>.sh</code>	docker-shell.sh, check-lidar.sh	Bash	Automate repeatable terminal procedures
<code>package.xml</code>	One per ROS package	ROS tools, rosdep, colcon	Package identity and dependencies
<code>CMakeLists.txt</code>	C++ ROS packages	CMake via colcon	Compile, link and install C++ packages
<code>setup.py / setup.cfg</code>	Python ROS packages	Python packaging via colcon	Install Python nodes and resources
<code>.cpp / .h / .hpp</code>	Unitree driver and SDK	C++ compiler	Compiled node implementation
<code>.py</code>	l1_monitor.py	Python interpreter	Python node or utility logic
<code>.launch.py</code>	unitree_l1.launch.py	ROS 2 launch	Start and connect several processes
<code>.yaml</code>	unitree_l1.yaml, octomap.yaml	ROS nodes or colcon	Structured runtime/build parameters
<code>.rviz</code>	unitree_l1.rviz	RViz2	Displays, topics, QoS, colours and camera
<code>.repos</code>	dependencies.repos	vcs import	Pin external Git repositories and commits
<code>Tests</code>	test_*.py	pytest / colcon test	Verify launch and configuration contracts
<code>Logs</code>	logs/tests, ros2 logs	Humans and debugging tools	Preserve diagnostic evidence
<code>ROS bags</code>	bags/*	rosbag2	Record and replay ROS messages
<code>.bt / .ot</code>	maps/my_room_01.bt	OctoMap tools	Store generated 3-D occupancy maps
<code>build / install / log</code>	colcon output directories	colcon and ROS	Generated compilation and installed overlay
<code>.pyc / __pycache__</code>	Python cache	Python runtime	Automatically generated bytecode cache

The detailed sections below explain when each file is edited, which program reads it and whether a rebuild is required.

3. Documentation and engineering evidence

These files explain, justify and preserve the project. They do not directly control the LiDAR.

3.1. Markdown documentation

.md - Human-readable project documentation

Examples: README.md, configuration-log.md, versions-lock.md, decisions.md, validation-matrix.md, hardware-runbook.md

- README.md introduces the project and its basic commands.
- configuration-log.md preserves the chronology of commands, failures and corrections.
- versions-lock.md records exact images, tags and Git commits for reproducibility.
- decisions.md explains architectural choices and rejected unsafe alternatives.
- validation-matrix.md records tests, methods, results and current verdicts.
- hardware-runbook.md documents electrical connections and safe operation.

Practical rule

Edit rule: Edit these when procedures, decisions, versions or test evidence change. Rebuild/reload: No ROS rebuild. Commit the documentation change to Git.

3.2. PDF documentation

.pdf - Fixed, printable publication format

Examples: Engineering manual, command journal, reports

- Keeps pagination and layout stable across computers.
- Useful for laboratory procedures, reporting, archiving and sharing.
- Can preserve screenshots, tables and signed or final versions.

Practical rule

Edit rule: Normally regenerate the PDF from its editable source instead of editing the PDF directly. Rebuild/reload: No ROS rebuild.

3.3. Raw validation logs

logs/, .log, .txt - Diagnostic evidence

Examples: logs/tests/, colcon logs, captured topic-rate output

- Preserve exact command output from builds, tests and hardware validation.
- Help distinguish a reproducible failure from a remembered description.
- Are evidence, not configuration.

Practical rule

Edit rule: Do not hand-edit raw evidence. Add a new log for a new validation run. Rebuild/reload: No ROS rebuild.

4. Docker environment and automation

These files define where ROS runs, which host resources enter the container and how repetitive operations are automated.

4.1. Docker image recipe

Dockerfile - Build instructions for the project image

Examples: docker/Dockerfile

- Selects the Ubuntu 22.04 / ROS 2 Humble base image.
- Installs compilers, colcon, rosdep, vcs, PCL, RViz2, TF2, rosbag2 and OctoMap dependencies.
- Creates the non-root ros user.
- Sets the working directory and container entry point.

Practical rule

Edit rule: Edit only when the software environment or installed system dependencies must change.
Rebuild/reload: Rebuild the Docker image, then recreate the container.

4.2. Docker Compose configuration

compose*.yaml - Runtime description for Docker Compose

Examples: docker/compose.yaml, compose.gui.yaml, compose.lidar.yaml

- compose.yaml defines the base image, bind mount, working directory, user and ROS_DOMAIN_ID.
- compose.gui.yaml adds DISPLAY, Xauthority, X11 socket and GPU/DRI access for RViz.
- compose.lidar.yaml maps one selected host serial device to /dev/unitree_lidar and sets LIDAR_PORT.
- The layers are merged so GUI and hardware access remain explicit and minimal.

Practical rule

Edit rule: Edit when changing mounts, environment variables, device access or container security.
Rebuild/reload: Recreate the container. An image rebuild is needed only if the Dockerfile also changed.

4.3. Shell automation scripts

.sh - Executable Bash procedures

Examples: docker-shell.sh, docker-build.sh, check-lidar.sh, gui-smoke-test.sh, assert-ros-container.sh, entrypoint.sh

- Turn a long sequence of terminal commands into one inspectable and repeatable operation.
- Can validate preconditions and stop when hardware selection is ambiguous or unsafe.
- The entrypoint initializes the normal container startup environment.
- A shell opened later with docker exec does not rerun the entrypoint, so ROS must be sourced again.

Practical rule

Edit rule: Edit when the automated procedure itself changes. Review carefully because scripts can modify system state. Rebuild/reload: Usually no colcon rebuild; restart or recreate the relevant shell/container.

5. ROS package and build definitions

These files allow colcon and ROS 2 to discover, build, install and run the packages.

5.1. ROS package manifest

package.xml - Identity card and dependency list for a ROS package

Examples: One package.xml inside l1_bringup, l1_monitor, l1_octomap_bringup and other ROS packages

- Declares package name, version, description, maintainer and licence.
- Declares build, runtime and test dependencies.
- Tells ROS tools which build system the package uses.
- rosdep uses these dependency declarations to install missing system packages.

Practical rule

Edit rule: Edit when adding/removing dependencies, changing metadata or changing the package interface.

Rebuild/reload: Rebuild the package and source install/setup.bash.

5.2. CMake build instructions

CMakeLists.txt - Build and install rules for C++ or ament_cmake packages

Examples: CMakeLists.txt in the Unitree driver and CMake-based packages

- Finds required libraries and ROS packages.
- Compiles .cpp files into executables or libraries.
- Links PCL, Boost, ROS interfaces and vendor SDK libraries.
- Installs executables, headers, launch files and configuration resources.
- Registers tests when required.

Practical rule

Edit rule: Edit when changing compilation, dependencies, targets or installed resources. Rebuild/reload: Run colcon build for the affected package and re-source the overlay.

5.3. Python package installation files

setup.py / setup.cfg - Install rules for Python ROS packages

Examples: setup.py and setup.cfg in Python packages such as a monitor package

- Declares Python modules and package metadata.
- Registers console scripts so ros2 run can find a Python executable.
- Installs launch, YAML and other package resources when configured.

Practical rule

Edit rule: Edit when adding Python nodes, entry points or installed data files. Rebuild/reload: Rebuild the package and source install/setup.bash.

5.4. colcon workspace defaults

colcon_defaults.yaml - Default options for colcon

Examples: Workspace-local colcon_defaults.yaml

- Restricts package discovery to the intended ROS 2 package directories.
- Prevents the ROS 1/catkin and raw CMake parts of the vendor repository from being discovered incorrectly.
- Enables --symlink-install for convenient development of Python, launch and configuration files.

Practical rule

Edit rule: Edit only when the set of buildable packages or default colcon behaviour changes. Rebuild/reload:

The next colcon command reads the new defaults; rebuild as appropriate.

5.5. Pinned repository manifest

.repos - External source checkout definition

Examples: config/dependencies.repos

- Uses YAML syntax but is interpreted by vcs import.
- Stores repository URLs and the desired branch, tag or exact commit.
- Makes the Unitree and OctoMap source checkouts reproducible.

Practical rule

Edit rule: Edit only when intentionally upgrading or replacing an external dependency. Rebuild/reload: Re-import/checkout the dependency, rebuild the workspace and revalidate.

6. Runtime source code and orchestration

These files contain the actual behavior or decide how several processes are started together.

6.1. C++ implementation files

.cpp / .cc / .cxx - Compiled node and SDK implementation

Examples: Unitree driver source, SDK examples and OctoMap C++ nodes

- Open and communicate with the serial device through the Unitree SDK.
- Convert measurements into sensor_msgs/msg/PointCloud2 and sensor_msgs/msg/Imu.
- Publish ROS topics and implement callbacks, services or algorithms.
- Run quickly because they are compiled into native executables.

Practical rule

Edit rule: Edit when changing the driver or algorithm itself, not just its parameter values. Rebuild/reload: Rebuild the affected package and restart the node.

6.2. C++ header files

.h / .hpp - Declarations shared by C++ source files

Examples: unitree_lidar_ros2.h, unitree_lidar_sdk.h

- Declare classes, methods, member variables, constants and data structures.
- Define interfaces that the corresponding .cpp files implement.
- May contain default parameter declarations or templates.

Practical rule

Edit rule: Edit when changing C++ interfaces or class structure. Rebuild/reload: Rebuild every package affected by the header change.

6.3. Python source files

.py - Python node, tool or utility implementation

Examples: l1_monitor.py and project utilities

- Implement ROS nodes with rclpy.
- Can subscribe to cloud and IMU topics, calculate rates and publish diagnostics.
- Are interpreted rather than compiled to native machine code.

Practical rule

Edit rule: Edit when changing Python behavior. Rebuild/reload: With --symlink-install, existing Python source changes may be visible immediately; restart the node. Rebuild for new installed files or metadata changes.

6.4. ROS 2 launch files

.launch.py - System-level process orchestration

Examples: unitree_l1.launch.py, l1_octomap.launch.py, unitree_l1_octomap.launch.py, view_saved_octomap.launch.py

- Declare launch arguments such as port, rviz, static_sensor and resolution.
- Find installed package resources.
- Load YAML parameter files and apply command-line overrides.
- Start the driver, monitor, TF publisher, OctoMap server and RViz.
- Remap topics and conditionally start processes.

Practical rule

Edit rule: Edit when changing which processes start, how they are connected or which values are exposed as launch arguments. Rebuild/reload: Rebuild the bringup package when required, source the overlay and restart the launch.

Launch versus node

ros2 run starts one executable. ros2 launch interprets a launch file that may start and connect several executables with parameters, remappings and conditions.

7. Configuration and visualisation files

These files normally change behavior without changing the implementation code.

7.1. ROS parameter files

.yaml - Structured parameters for ROS nodes

Examples: config/unitree_l1.yaml and config/octomap.yaml

- unitree_l1.yaml configures the serial port, range conversion, cloud/IMU topics, frames, scan accumulation and monitor thresholds.
- octomap.yaml configures the world frame, voxel resolution, maximum range, hit/miss probabilities, filters and map output behavior.
- The top-level YAML key must match the effective node name for the parameters to apply.
- Launch arguments or command-line -p values can override YAML values.

Practical rule

Edit rule: Edit parameters here when changing normal operating values. Rebuild/reload: Existing linked config changes may be visible with --symlink-install, but stop and relaunch the node; rebuild if a new installed file or install rule changed.

Typical precedence for the LiDAR port:
 explicit port:=... launch argument
 > LIDAR_PORT environment variable
 > YAML or driver fallback

7.2. RViz profiles

.rviz - Saved visualisation layout and display settings

Examples: unitree_l1.rviz, l1_octomap.rviz, saved_octomap.rviz

- Store the Fixed Frame, PointCloud2 and MarkerArray displays.
- Store topic names, QoS compatibility, colour transformer, rainbow mode, point size and decay time.
- Store grid, axes, camera position and which displays are enabled.
- Do not contain the actual LiDAR measurements or OctoMap.

Practical rule

Edit rule: Edit through RViz and save the profile, or edit carefully as structured text. Rebuild/reload: No algorithm rebuild; ensure the profile is installed or open it directly from src.

7.3. Launch and configuration tests

test_*.py / *_test.py - Automated regression checks

Examples: Package tests for launch/config contracts

- Check that launch files import and expose the expected arguments.
- Check that configuration files exist and use intended values.
- Can verify that a combined launch starts exactly one RViz process.
- Prevent a previously fixed integration error from returning.

Practical rule

Edit rule: Edit when adding a requirement or correcting the expected behavior. Rebuild/reload: Run colcon test and inspect colcon test-result --verbose.

8. Recorded and generated data

These files are outputs of experiments or build processes. They are not the authoritative source code.

8.1. ROS bag recordings

bag directory / rosbag2 files - Recorded ROS message streams

Examples: bags/ containing /unilidar/cloud, /unilidar/imu, /tf and /tf_static

- Record timestamped messages from live ROS topics.
- Allow replay without reconnecting the physical LiDAR.
- Support debugging, algorithm development and repeatable demonstrations.

Practical rule

Edit rule: Do not text-edit bag storage. Record a new bag or use rosbag2 tools. Rebuild/reload: No rebuild.

8.2. OctoMap map files

.bt / .ot - Persistent 3-D occupancy map data

Examples: maps/my_room_01.bt and optional .ot files

- .bt is a compact binary occupancy-tree representation.
- .ot stores a fuller OctoMap serialization and is usually larger.
- They can be reopened later without the LiDAR.
- They are generated data and should not be edited as text.

Practical rule

Edit rule: Create a new map with the OctoMap saver rather than modifying binary data manually.
Rebuild/reload: No rebuild.

8.3. colcon-generated directories

build/ install/ log/ - Compilation products and installed workspace overlay

Examples: ros2_ws/build, ros2_ws/install, ros2_ws/log

- build/ contains temporary compiler and CMake files.
- install/ contains the executables, libraries, launch/config resources and setup.bash used by ROS.
- log/ contains colcon build and test logs.
- The authoritative editable source remains under src/.

Practical rule

Edit rule: Do not manually edit build/ or install/. Edit src/ and let colcon regenerate them. Rebuild/reload: Run colcon build, then source install/setup.bash.

8.4. Python cache files

.pyc / __pycache__ - Automatically generated Python bytecode cache

Examples: __pycache__/unitree_l1.launch.cpython-*.pyc

- Speed up later Python imports.
- Are regenerated automatically.
- Are not authoritative project source and should not be committed or edited.

Practical rule

Edit rule: Ignore or delete the cache when debugging; edit the corresponding .py file instead. Rebuild/reload: No rebuild is normally required; restart Python processes if needed.

9. Which file should I edit?

Use this decision table before changing the project.

Goal	Primary file(s)	Typical next action
Change LiDAR port, topics, frames or distance limits	unitree_l1.yaml or launch argument	Stop and relaunch; query effective parameters
Change OctoMap resolution or maximum range	octomap.yaml or launch argument	Restart octomap_server and create a new map
Change point colours, size, QoS or camera	*.rviz	Save profile and reopen RViz
Start an extra node or change launch order	*.launch.py	Rebuild bringup package if needed; re-source and relaunch
Change LiDAR decoding or published message logic	.cpp / .h	Rebuild driver package and restart
Change monitor calculations	l1_monitor.py	Restart node; rebuild if installation metadata changed
Add a ROS dependency	package.xml plus CMakeLists.txt/setup.py	Install dependencies, rebuild and re-source
Add a package to the workspace build	colcon_defaults.yaml	Run colcon list, then build
Change container packages	Dockerfile	Rebuild image and recreate container
Change serial/GPU mounts or runtime environment	compose*.yaml or docker-shell.sh	Recreate container
Upgrade Unitree or OctoMap source version	dependencies.repos and versions-lock.md	Import checkout, rebuild and validate
Preserve a sensor experiment	ROS bag	Record with rosbag2
Preserve a 3-D occupancy map	.bt / .ot	Use octomap_saver_node
Document a correction or decision	docs/*.md	Commit the documentation update

10. Safe editing workflow

1. Identify which program reads the file: Docker, colcon, ROS launch, a ROS node, RViz, vcs or a human.
2. Edit the authoritative source under the project or ros2_ws/src/, not a generated copy under build/ or install/.
3. Build only what is necessary: a C++/metadata change needs colcon; a Dockerfile change needs an image rebuild.
4. Source /opt/ros/humble/setup.bash and install/setup.bash in each newly attached Docker shell.
5. Stop and relaunch nodes because most parameters are read when a node starts.
6. Verify the effective runtime state with ros2 param get, ros2 node list, ros2 topic info -v and ros2 topic hz.
7. Record the successful change in the appropriate documentation and tests.

Do not confuse source and installed copies

Edit /workspace/ros2_ws/src/... . The /workspace/ros2_ws/install/... tree is generated by colcon and is what ROS runs after sourcing the overlay.

```
Authoritative source: /workspace/ros2_ws/src/...
Generated build files: /workspace/ros2_ws/build/...
Installed ROS overlay: /workspace/ros2_ws/install/...
Build/test logs:      /workspace/ros2_ws/log/...
```

11. Compact glossary

Source code	Files that implement behavior, principally .cpp, .h/.hpp and ordinary .py files.
Manifest	A file that declares identity, dependencies or external sources, such as package.xml or dependencies.repos.
Build system	The tools that transform source into installed executables and resources: colcon, CMake and Python packaging.
Launch file	A ROS 2 orchestration program that starts and configures several processes.
Parameter file	A structured file whose values are loaded into a running node.
Overlay	The installed workspace added on top of /opt/ros/humble by sourcing install/setup.bash.
Generated artifact	A file produced by a build or experiment, such as install/, logs, bags or maps.
Pinned dependency	An external repository fixed to a known tag or commit for reproducibility.

Final principle

Code defines capability.
 Configuration selects behavior.
 Launch files assemble the system.
 Build files make it executable.
 Generated data proves what happened.

Reference basis: Unitree L1 engineering manual - Docker, RViz2 and OctoMap workflow, assembled July 2026.